

UNIT-V: File System

- Different Types of Files
- Access methods
- Directory Structures
- Various Allocation methods
- Disk Scheduling and Management
 - Disk Scheduling Algorithms
 - Example-1
 - Example-2
 - FCFS
 - SSTF
 - SCAN
 - C-SCAN
 - LOOK / C-LOOK
 - Selecting a Disk Scheduling Algorithm
- Introduction to Distributed File System (DFS)
- Exam Questions

File Overview

- A file is a collection of similar records. The file is treated as a single entity by users and applications
- A file is a container for a collection of information.
- File represents Programs and Data.
- Data files may be numeric, alphabetic, binary or alpha numeric
- In general, file is sequence of bits, bytes, lines or records.

Contd..

- A file has a certain defined structure according to its type.
 1. Text file
 2. Source file
 3. Executable file
 4. Object file

Contd..

- A text file is a sequence of characters organized into lines.
- A source file is a sequence of subroutine and function.
- An Object file is a sequence of bytes organized into blocks understandable by the systems linker.
- An executable file is a series of code sections that the loader can bring into memory and execute.

File Attributes

- **Name**
 - Only information kept in human-readable form
- **Identifier**
 - Unique tag (number) identifies file within file system
- **Type**
 - Needed for systems that support different types
- **Location**
 - Pointer to file location on device
- **Size**
 - Current file size
- **Protection**
 - Controls who can do reading, writing, executing
- **Time, date, and user identification**
 - Data for protection, security, and usage monitoring

All Information about files is

- Kept in the directory structure
- Maintained on the disk

File Operations

- **Create**
- **Write**
- **Read**
- **Reposition within file**
- **Delete**
- **Truncate**

File Types – Name, Extension

file type	usual extension	function
executable	exe, com, bin or none	ready-to-run machine-language program
object	obj, o	compiled, machine language, not linked
source code	c, cc, java, pas, asm, a	source code in various languages
batch	bat, sh	commands to the command interpreter
text	txt, doc	textual data, documents
word processor	wp, tex, rtf, doc	various word-processor formats
library	lib, a, so, dll	libraries of routines for programmers
print or view	ps, pdf, jpg	ASCII or binary file in a format for printing or viewing
archive	arc, zip, tar	related files grouped into one file, sometimes compressed, for archiving or storage
multimedia	mpeg, mov, rm, mp3, avi	binary file containing audio or A/V information

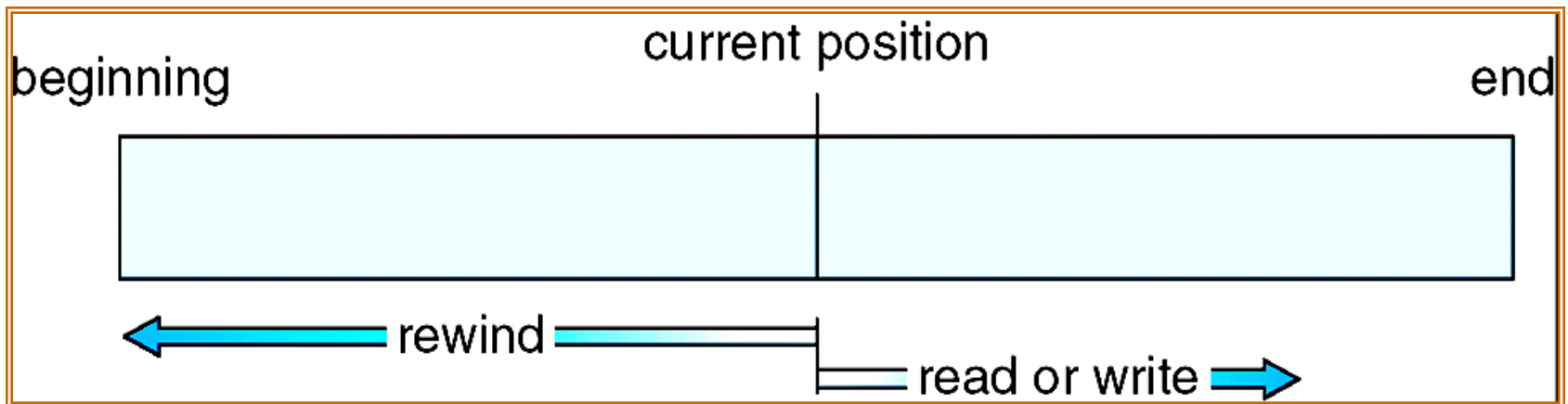


Access Methods

Sequential Access

- The simplest access method, in which the information in the file is processed in order, one record after the other.
- Editors and Compilers access files in this fashion.
- A read operation - **read next**- reads the next portion of the file and automatically advances a file pointer, which tracks I/O location.
- The write operation -**write next**- appends to the end of the file and advances to end of newly written material.
- **Reset**
- It is based on a tape model of a file

Sequential–Access File



Contd..

Direct Access:

- Another method is direct access (or relative access)
- A file is made up of fixed-length logical records that allows programs to read and write records rapidly in no particular order.
- The Direct –access method is based on a disk model of a file, since disk allow random access to any file block.
- For direct access, the file is viewed as numbered sequence of blocks or records.
- They are of great use for immediate access to large amounts of information.
- Databases are often of this type.

Contd..

Direct Access

read n

write n

position to n

read next

write next

rewrite n

n = relative block number

The block no. provided by user to OS is normally a **relative block number**.

A relative block no. is an index relative to the beginning of the file.

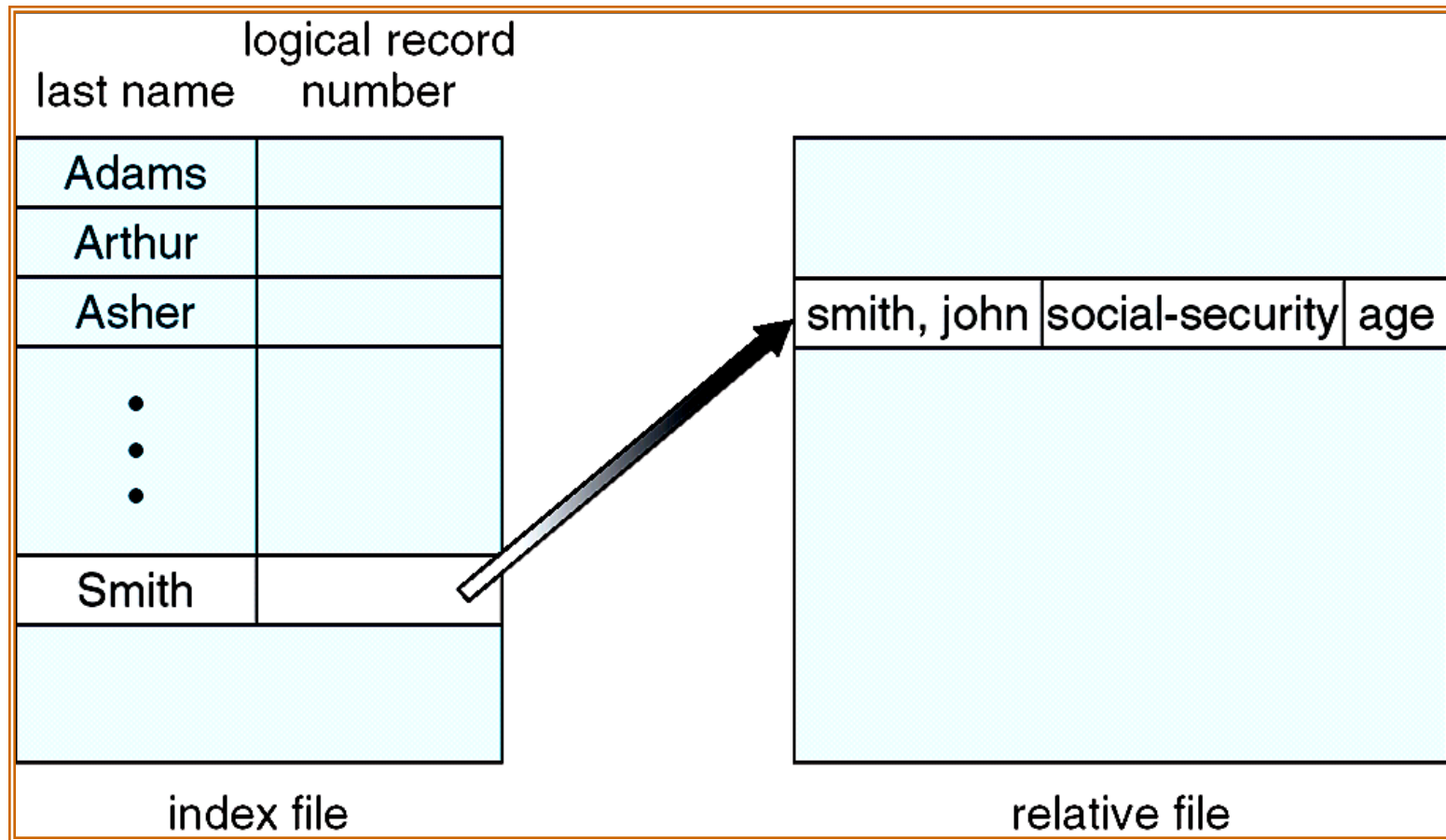
Simulation of Sequential Access on a Direct-Access File

sequential access	implementation for direct access
<i>reset</i>	<i>cp = 0;</i>
<i>read next</i>	<i>read cp;</i> <i>cp = cp + 1;</i>
<i>write next</i>	<i>write cp;</i> <i>cp = cp + 1;</i>

Other Access Methods:

- Other access methods can be built on top of a direct-access method. These methods involve the construction of an index for the file.
- The index, like an index in the back of a book, contains pointers to the various blocks.
- To find a record in the file, we first search the index and then use the pointer to access the file directly and to find the desired record.
- With large files, the index file itself may become too large to be kept in memory. One solution is to create an index for the index file.
- The primary index file would contain pointers to secondary index files, which would point to actual data items.

Example of Index and Relative Files



Various Allocation Methods

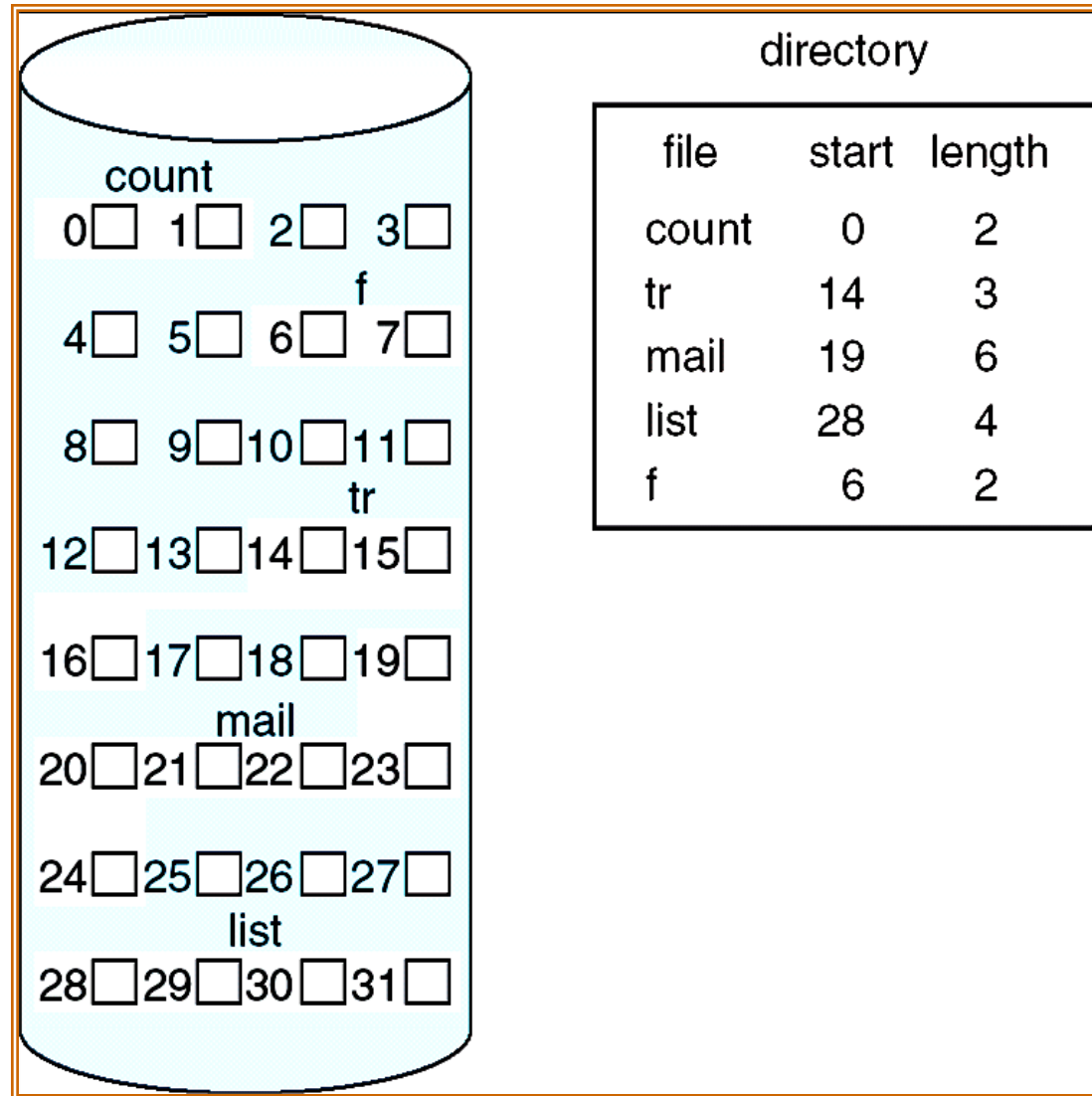
Refers to how disk blocks are allocated to files.

- **Contiguous allocation**
- **Linked allocation**
- **Indexed allocation**

Contiguous Allocation

- **Each file occupies a set of contiguous blocks on the disk.**
- **Simple**
 - **Only starting location (block #) and length (No. of blocks) are required.**
- **Random access.**
- **Wasteful of space (dynamic storage-allocation problem).**
- **Files cannot grow.**

Contiguous Allocation of Disk Space



- In this scheme, each file occupies a contiguous set of blocks on the disk.

For example, if a file requires n blocks and is given a block b as the starting location, then the blocks assigned to the file will be: $b, b+1, b+2, \dots, b+n-1$.

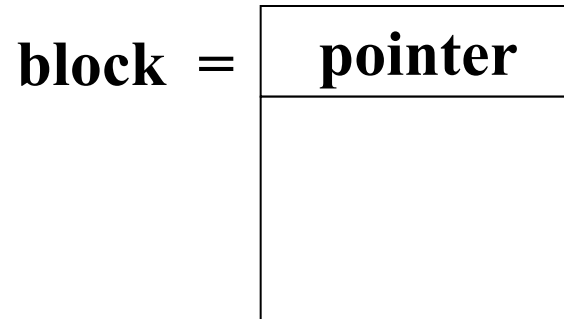
This means that given the starting block address and the length of the file (in terms of blocks required), we can determine the blocks occupied by the file.

The directory entry for a file with contiguous allocation contains address of starting block & length of the allocated portion.

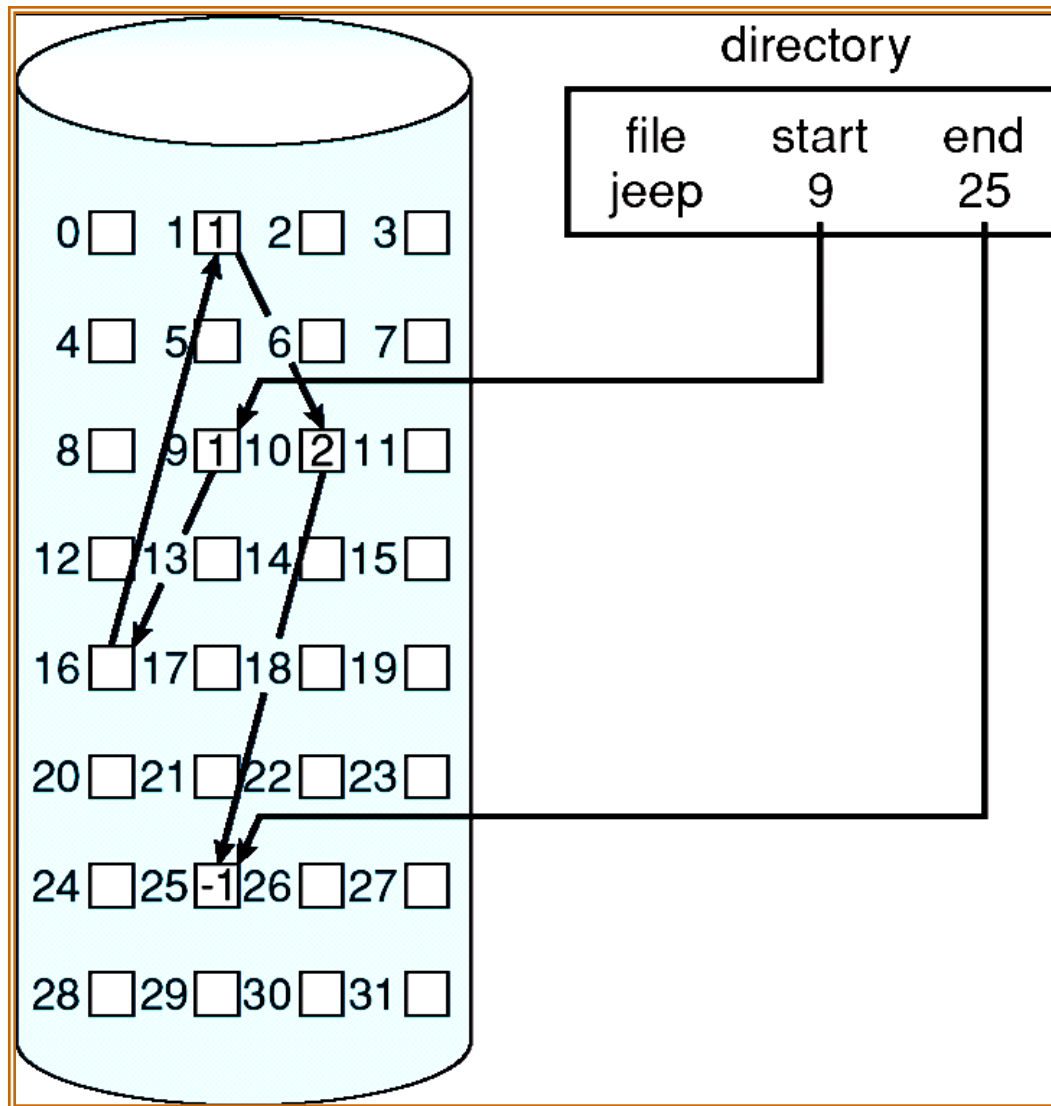
The *file 'mail'* in the following figure starts from the block 19 with length = 6 blocks. Therefore, it occupies 19, 20, 21, 22, 23, 24 blocks.

Linked Allocation

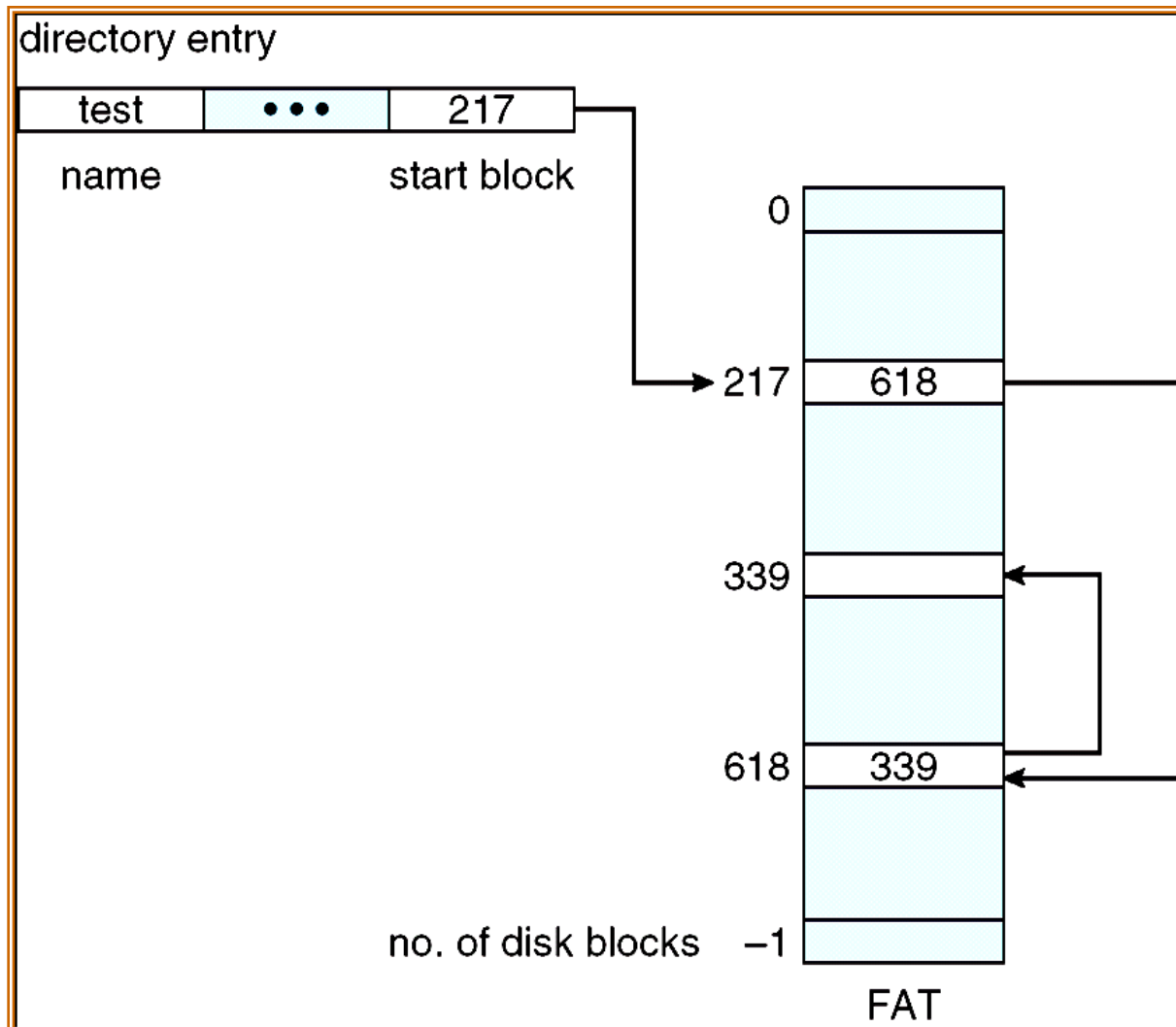
- **Each file is a linked list of disk blocks.**
 - **Blocks may be scattered anywhere on the disk.**



Linked Allocation

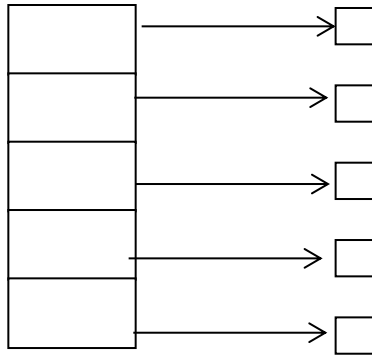


File-Allocation Table



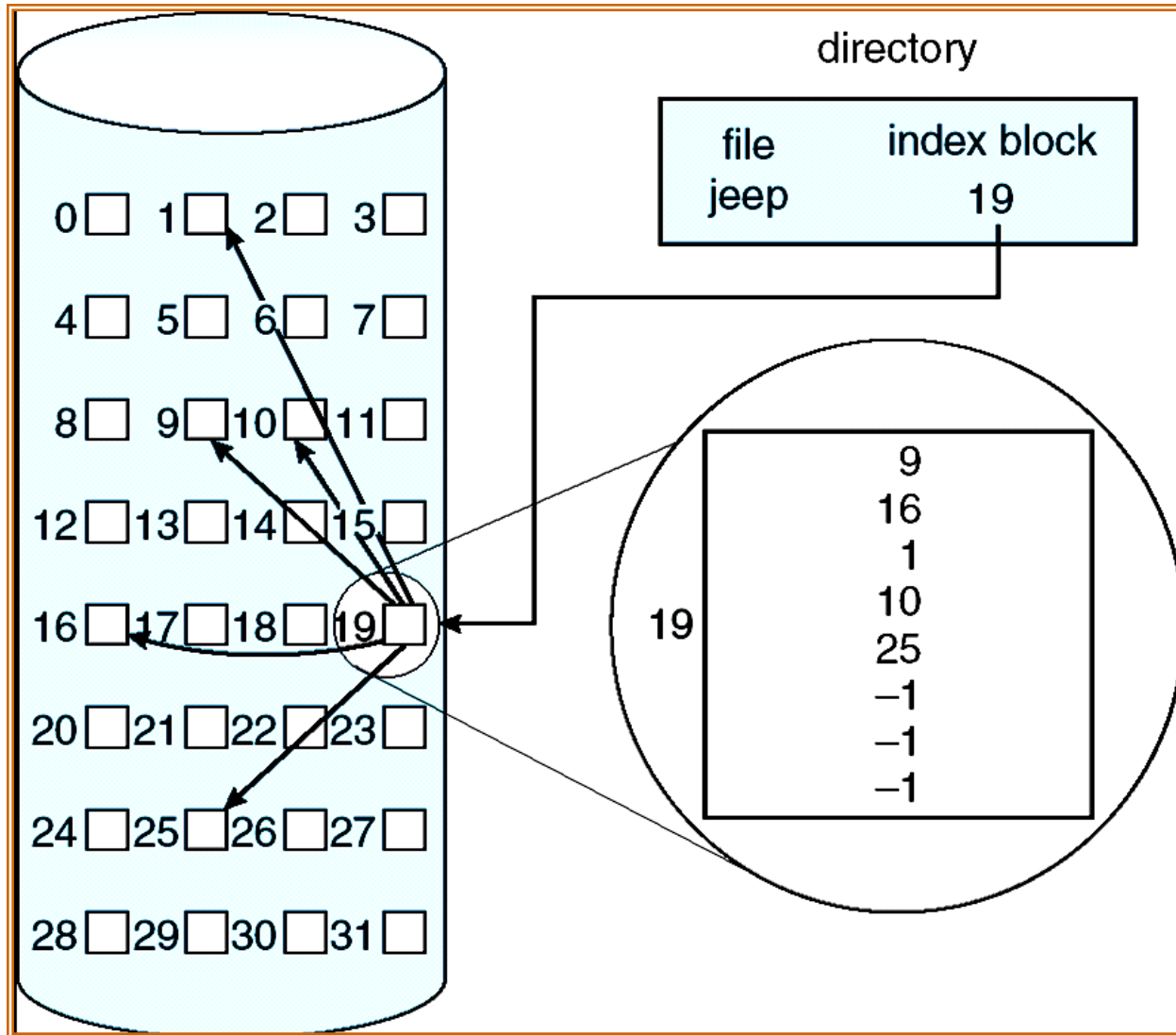
Indexed Allocation

- Brings all pointers together into the *index block*.
- Logical view.



index table

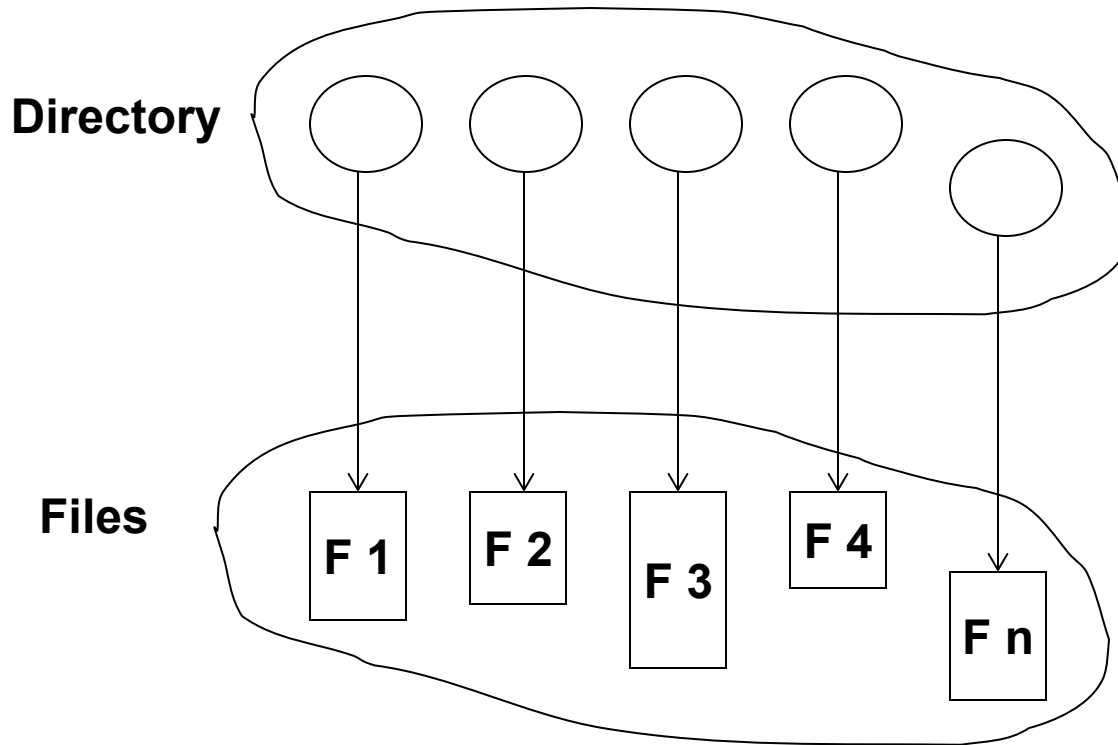
Example of Indexed Allocation



<<

Directory Structure

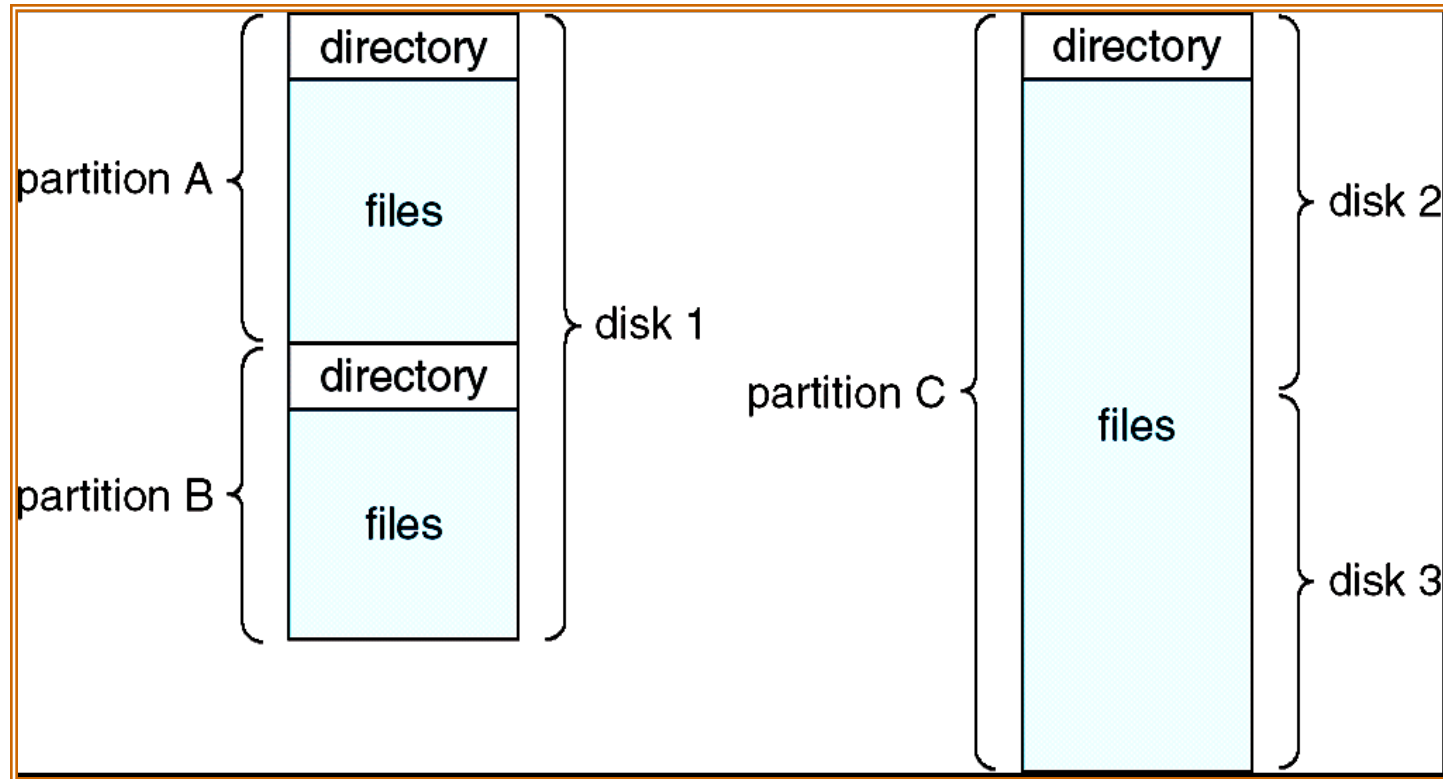
- A collection of nodes containing information about all files.



Both the directory structure and the files reside on disk.

Backups of these two structures are kept on tapes.

A Typical File–System Organization



Operations Performed on Directory

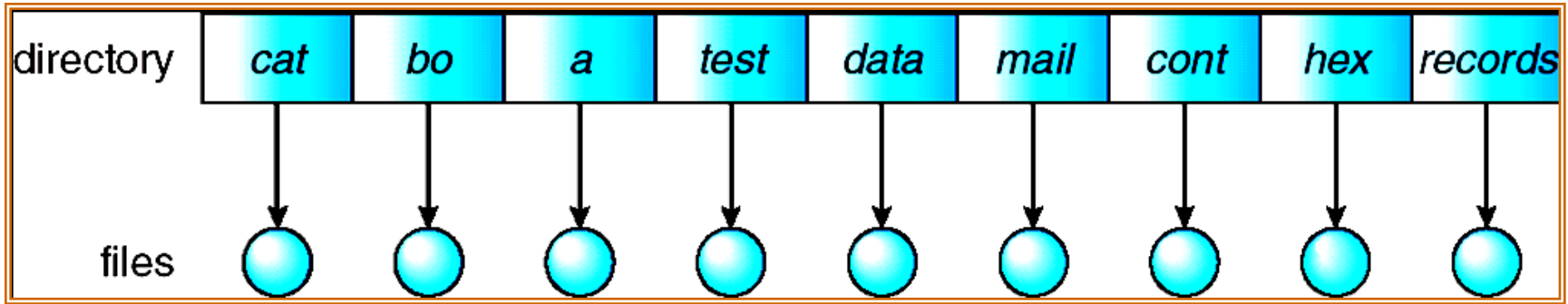
- *Search* for a file
- *Create* a file
- *Delete* a file
- *List* a directory
- *Rename* a file
- *Traverse* the file system

Organize the Directory (Logically) to Obtain

- **Efficiency**
 - Locating a file quickly
- **Naming**
 - Convenient to users
 - Two users can have same name for different files
 - The same file can have several different names
- **Grouping**
 - Logical grouping of files by properties
 - Examples
 - All Java programs
 - All games
 - ...

Single-Level Directory

- A single directory for all users



- Naming problem
- Grouping problem

This kind of Directory structure leads to confusion of file names among different users.

Advantages:

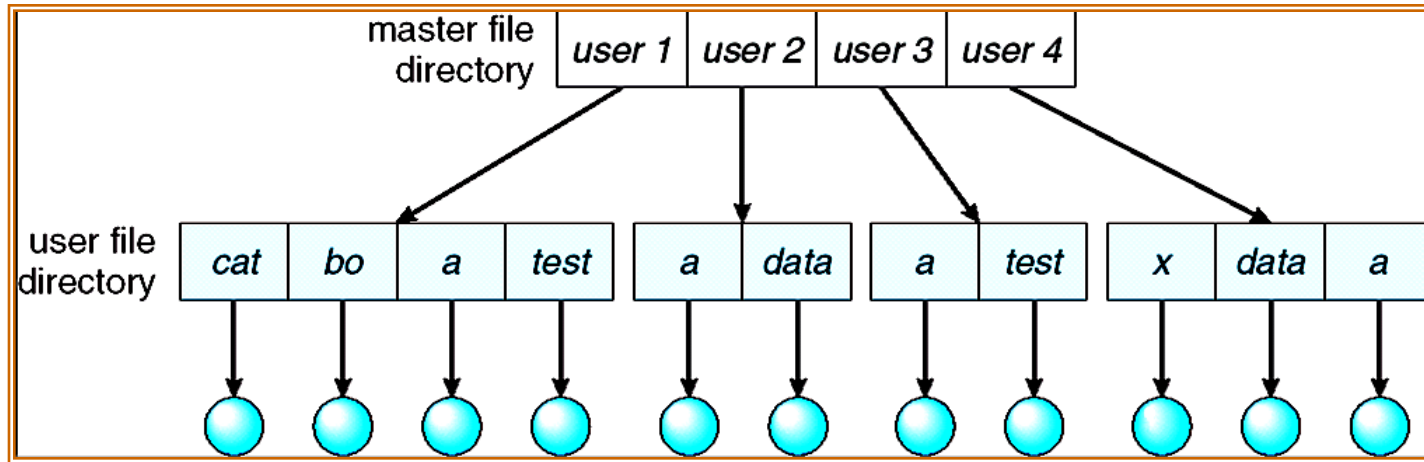
- Since it is a single directory, so its implementation is very easy.
- If the files are smaller in size, searching will become faster.
- The operations like file creation, searching, deletion, updating are very easy in such a directory structure.

Disadvantages:

- There may chance of name collision because two files can have the same name.
- Searching will become time taking if the directory is large.
- This can not group the same type of files together.

Two-Level Directory

- Separate directory for each user



- Path name
- Can have the same file name for different user
- Efficient searching
- No grouping capability

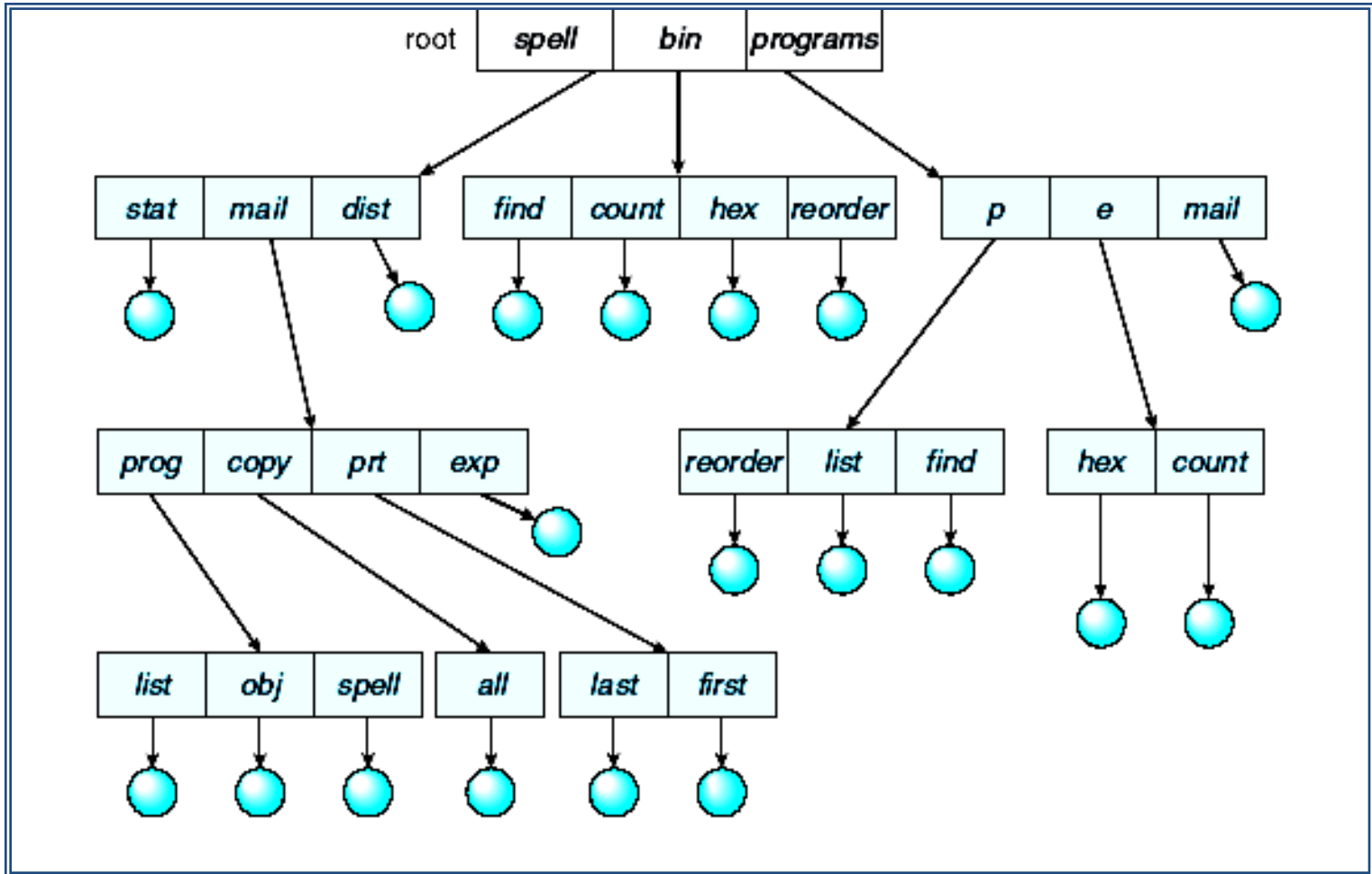
Advantages:

- We can give full path like /User-name/directory-name/.
- Different users can have the same directory as well as the file name.
- Searching of files becomes easier due to pathname and user-grouping.

Disadvantages:

- A user is not allowed to share files with other users.
- Still, it not very scalable, two files of the same type cannot be grouped together in the same user.

Tree-Structured Directories



Tree-Structured Directories (Cont)

- **Efficient Searching**
- **Grouping Capability**
- **Current Directory (Working Directory)**
 - **cd /spell/mail/prog**
 - **type list**

Tree-Structured Directories (Cont)

- ***Absolute Path name***: begins at the root and follows a path down to the specified file, giving the directory names on the path
- ***Relative path name***: defines the path from the current directory.

- **Creating a new file is done in current directory**
- **Delete a file**

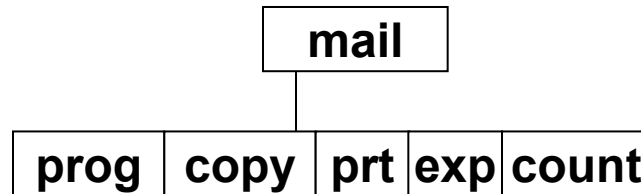
rm <file-name>

- **Creating a new subdirectory is done in current directory**

mkdir <dir-name>

Example: if in current directory **/mail**

mkdir count



Deleting “mail” ⇒ deleting the entire subtree rooted by “mail”

Advantages:

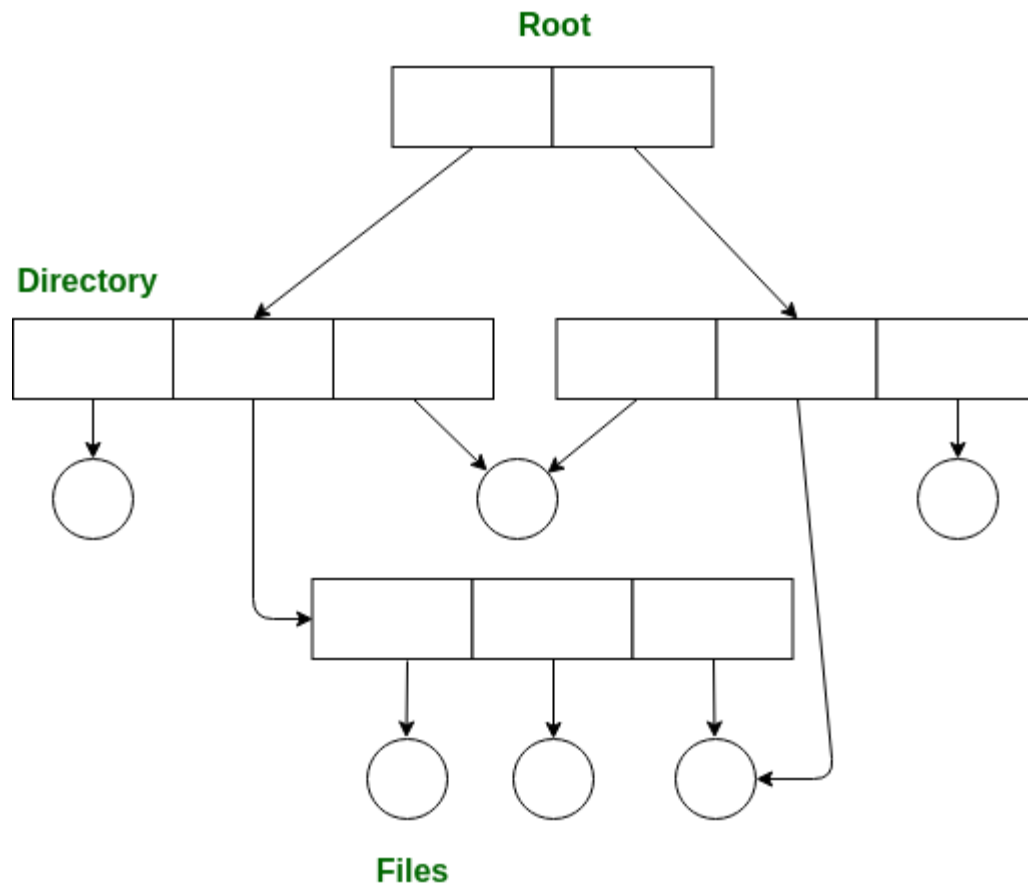
- Very general, since full pathname can be given.
- Very scalable, the probability of name collision is less.
- Searching becomes very easy, we can use both absolute paths as well as relative.

Disadvantages:

- Every file does not fit into the hierarchical model, files may be saved into multiple directories.
- We can not share files.
- It is inefficient, because accessing a file may go under multiple directories.

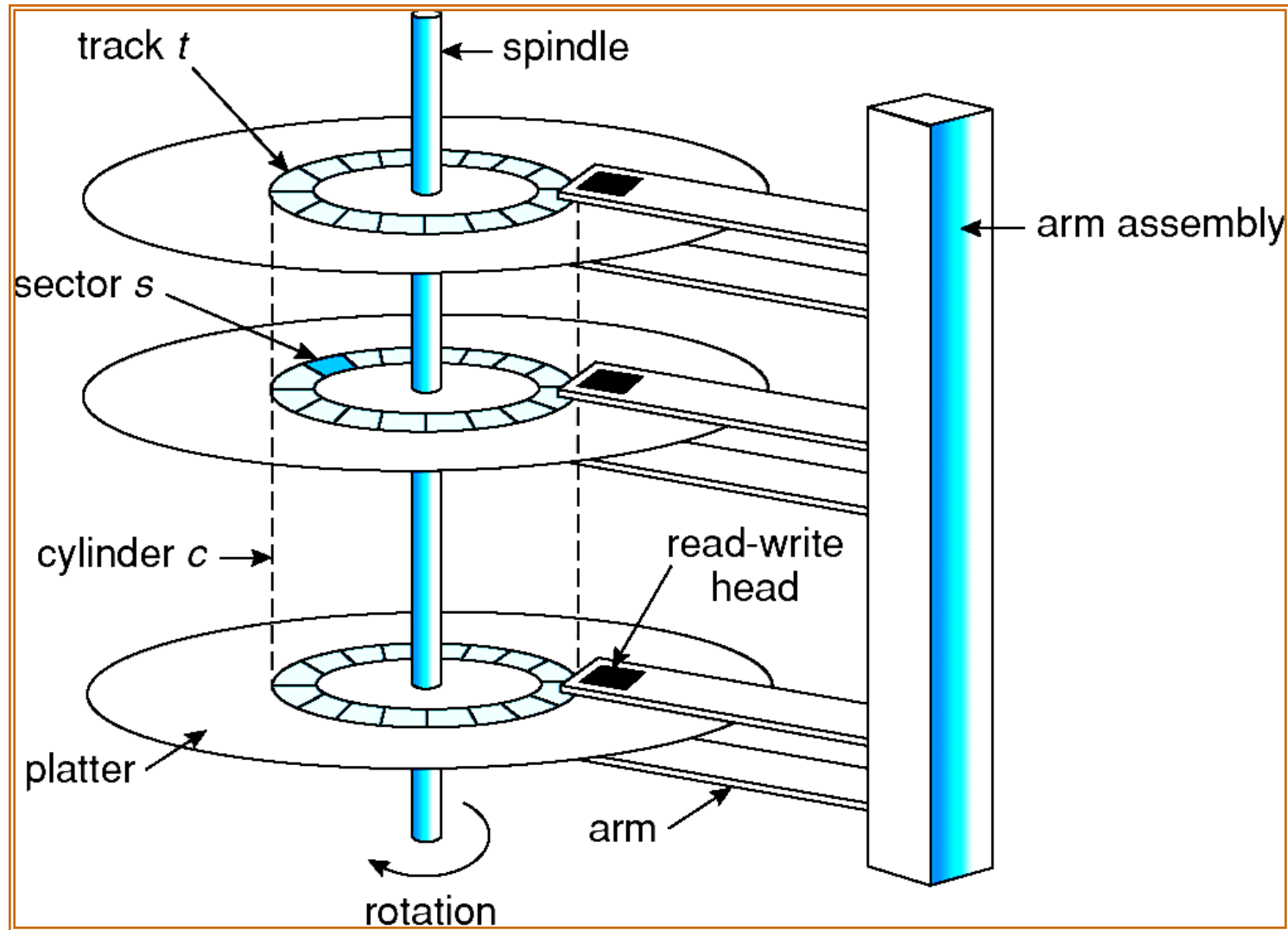
Acyclic Graph Directories

- It allows directories to have shared subdirectories and files
- Same file or directory may be in two different directories
- Graph with no cycles is a generalization of the tree structured subdirectory scheme
- Shared files and subdirectories can be implemented by using links.
- A link is effectively a pointer to another file or subdirectory.
- An acyclic graph directory structure is more flexible than is a simple tree structure, but some times it is more complex.
- Several problems must be considered carefully. A file may now have multiple absolute path names. Consequently, distinct file names may refer to the same file.(similar to aliasing in programming languages).
- Another problem involves deletion.



- **Advantages:**
- We can share files.
- Searching is easy due to different-different paths.
- **Disadvantages:**
- We share the files via linking, in case deleting it may create the problem,
- If the link is a soft link then after deleting the file we left with a dangling pointer.
- In the case of a hard link, to delete a file we have to delete all the references associated with it.

Disk Scheduling and Management



Moving Head Disk Mechanism

Disk Scheduling

- **Operating System**
 - Responsible for using Hardware efficiently
 - For Disk Drives
 - Fast Access Time
 - Disk Bandwidth
- **Components of Access time**
 - *Seek time*
 - Time for the Disk arm to move the Heads to the Cylinder containing the Desired Sector
 - *Rotational latency*
 - Additional time waiting for the Disk to Rotate the Desired sector to the Disk Head.
- **Minimize Seek time**
- **Seek time $\approx$ Seek distance**
- **Disk Bandwidth**
 - Total No. of bytes transferred divided by the total time between the first request for Service and the Completion of the Last transfer.



Disk Scheduling – Example 1

- 1. Suppose that a disk drive has 200 cylinders, numbered 0 to 199. The drive is currently serving a request at cylinder 53, and the previous request was at cylinder 45. The queue of pending requests in FIFO order is 98, 183, 37, 122, 14, 124, 65, 67. Starting from the current head position, what is the total distance (in cylinders) that the disk arm moves to satisfy all the pending requests, for each of the following *disk scheduling* algorithms?**
 - a. FCFS**
 - b. SSTF**
 - c. SCAN**
 - d. C-SCAN**
 - e. LOOK**
 - f. C-LOOK**



Disk Scheduling – Example 2

2. Suppose that a disk drive has 5000 cylinders, numbered 0 to 4999. The drive is currently serving a request at cylinder 143, and the previous request was at cylinder 125. The queue of pending requests in FIFO order is 86, 1470, 913, 1774, 948, 1509, 1022, 1750, 130. Starting from the current head position, what is the total distance (in cylinders) that the disk arm moves to satisfy all the pending requests, for each of the following *disk scheduling* algorithms?

- a. FCFS**
- b. SSTF**
- c. SCAN**
- d. C-SCAN**
- e. LOOK**
- f. C-LOOK**

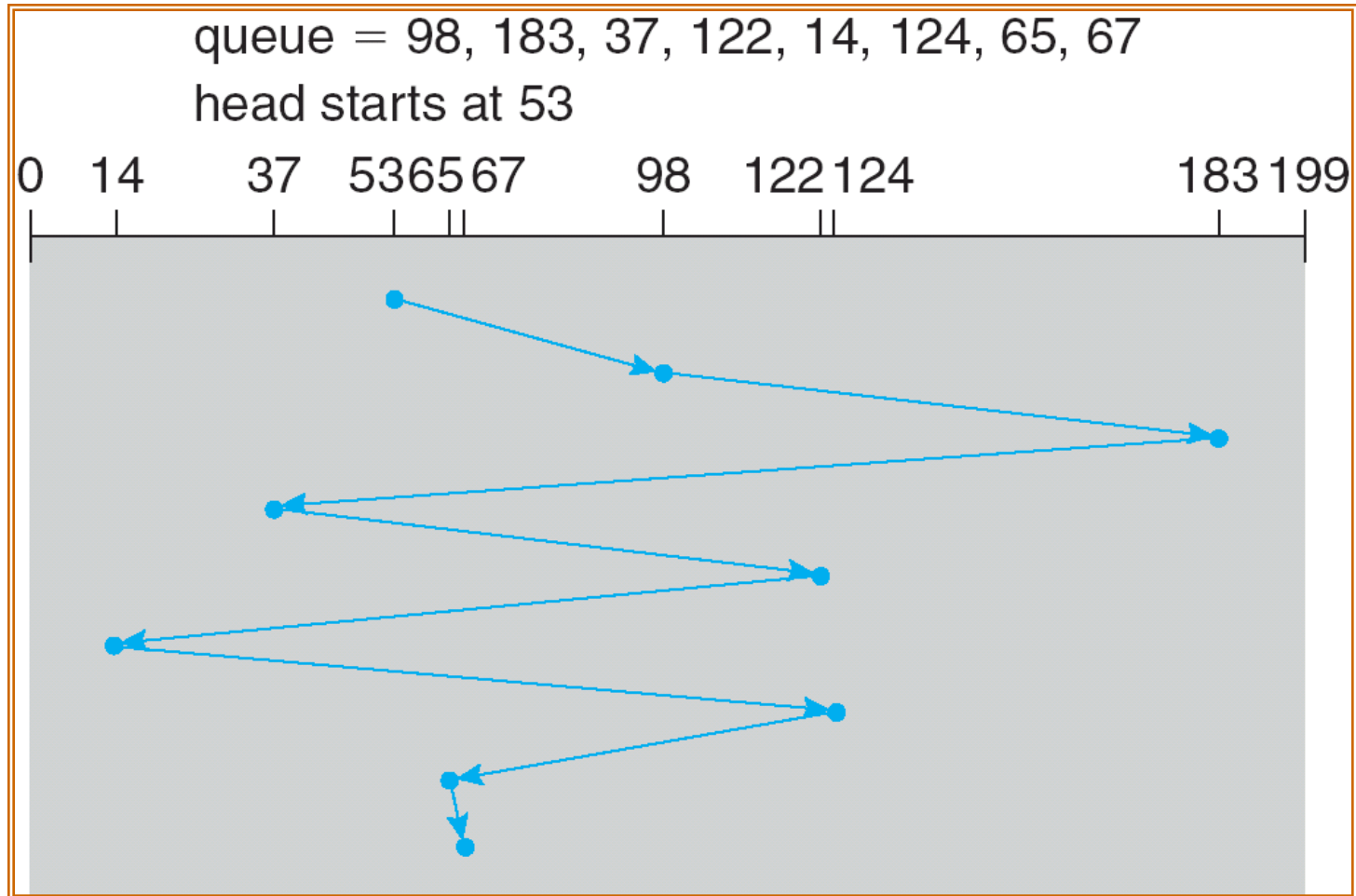


FCFS Disk Scheduling

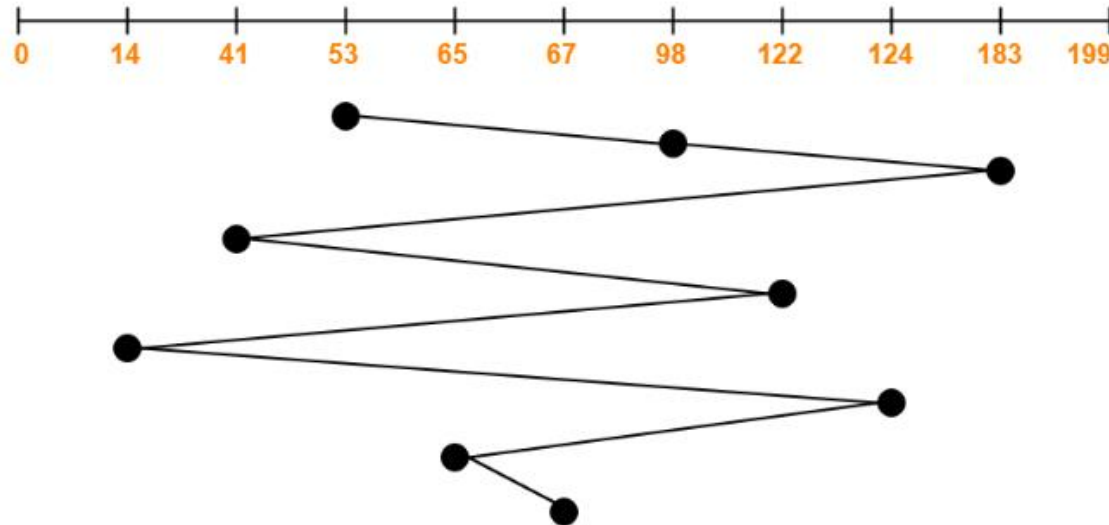
- Request queue (0-199).

98, 183, 37, 122, 14, 124, 65, 67

Head pointer 53



Consider a disk queue with requests for I/O to blocks on cylinders 98, 183, 41, 122, 14, 124, 65, 67. The **FCFS** scheduling algorithm is used. The head is initially at cylinder number 53 moving towards larger cylinder numbers on its servicing pass. The cylinders are numbered from 0 to 199. The total head movement (in number of cylinders) incurred while servicing these requests is-----



Total head movements incurred while servicing these requests

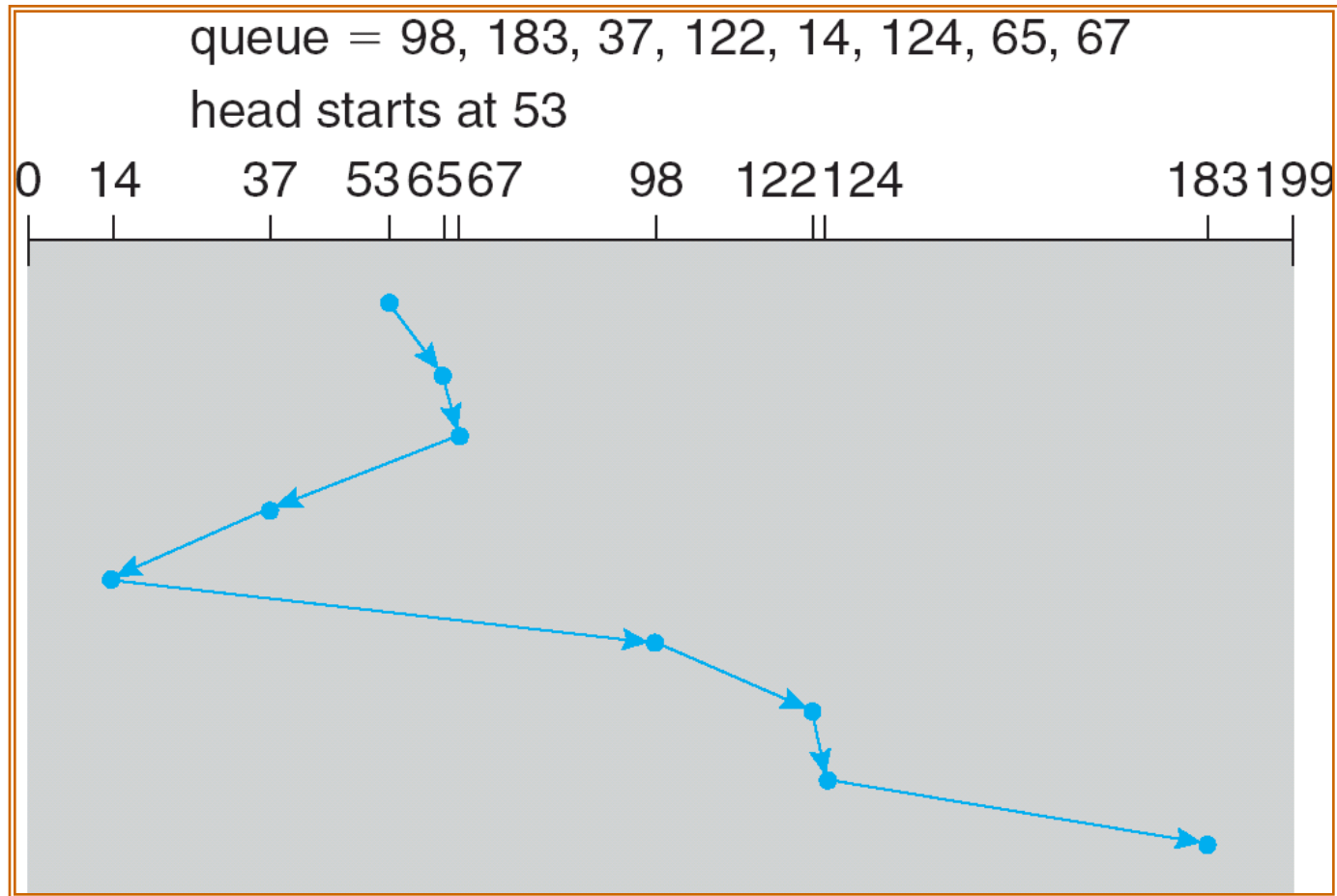
$$= (98 - 53) + (183 - 98) + (183 - 41) + (122 - 41) + (122 - 14) + (124 - 14) + (124 - 65) + (67 - 65)$$

$$= 45 + 85 + 142 + 81 + 108 + 110 + 59 + 2$$

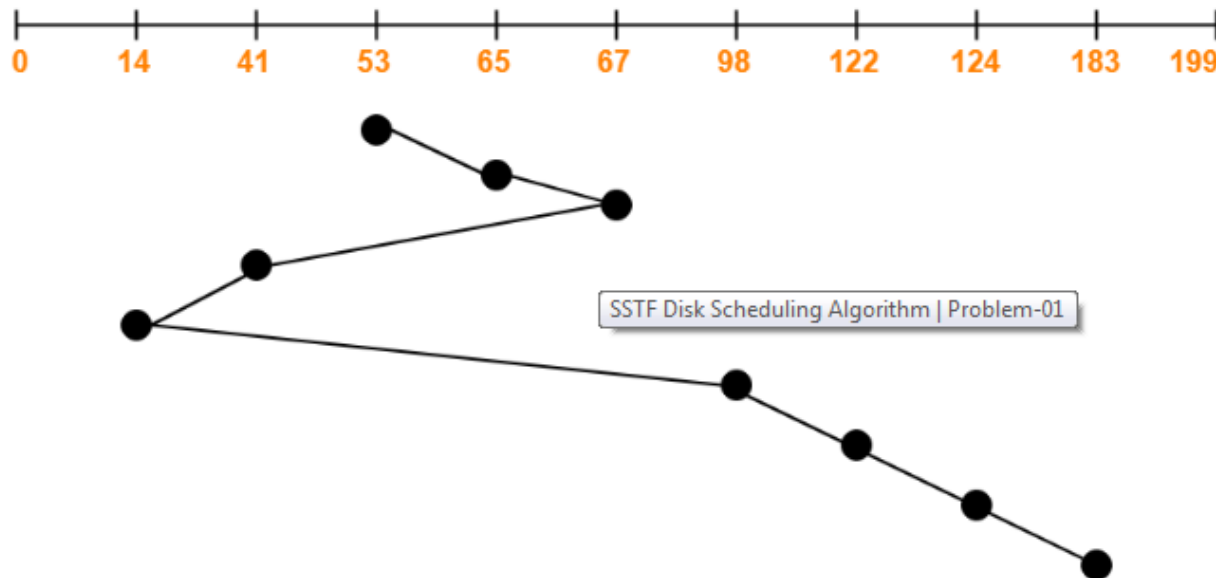
$$= 632$$

SSTF Disk Scheduling Algorithm

- Selects the request with the minimum seek time from the current head position.
- SSTF scheduling is a form of SJF scheduling; may cause starvation of some requests.



Consider a disk queue with requests for I/O to blocks on cylinders 98, 183, 41, 122, 14, 124, 65, 67. The **SSTF** scheduling algorithm is used. The head is initially at cylinder number 53 moving towards larger cylinder numbers on its servicing pass. The cylinders are numbered from 0 to 199. The total head movement (in number of cylinders) incurred while servicing these requests is-----



Total head movements incurred while servicing these requests

$$= (65 - 53) + (67 - 65) + (67 - 41) + (41 - 14) + (98 - 14) + (122 - 98) + (124 - 122) + (183 - 124)$$

$$= 12 + 2 + 26 + 27 + 84 + 24 + 2 + 59$$

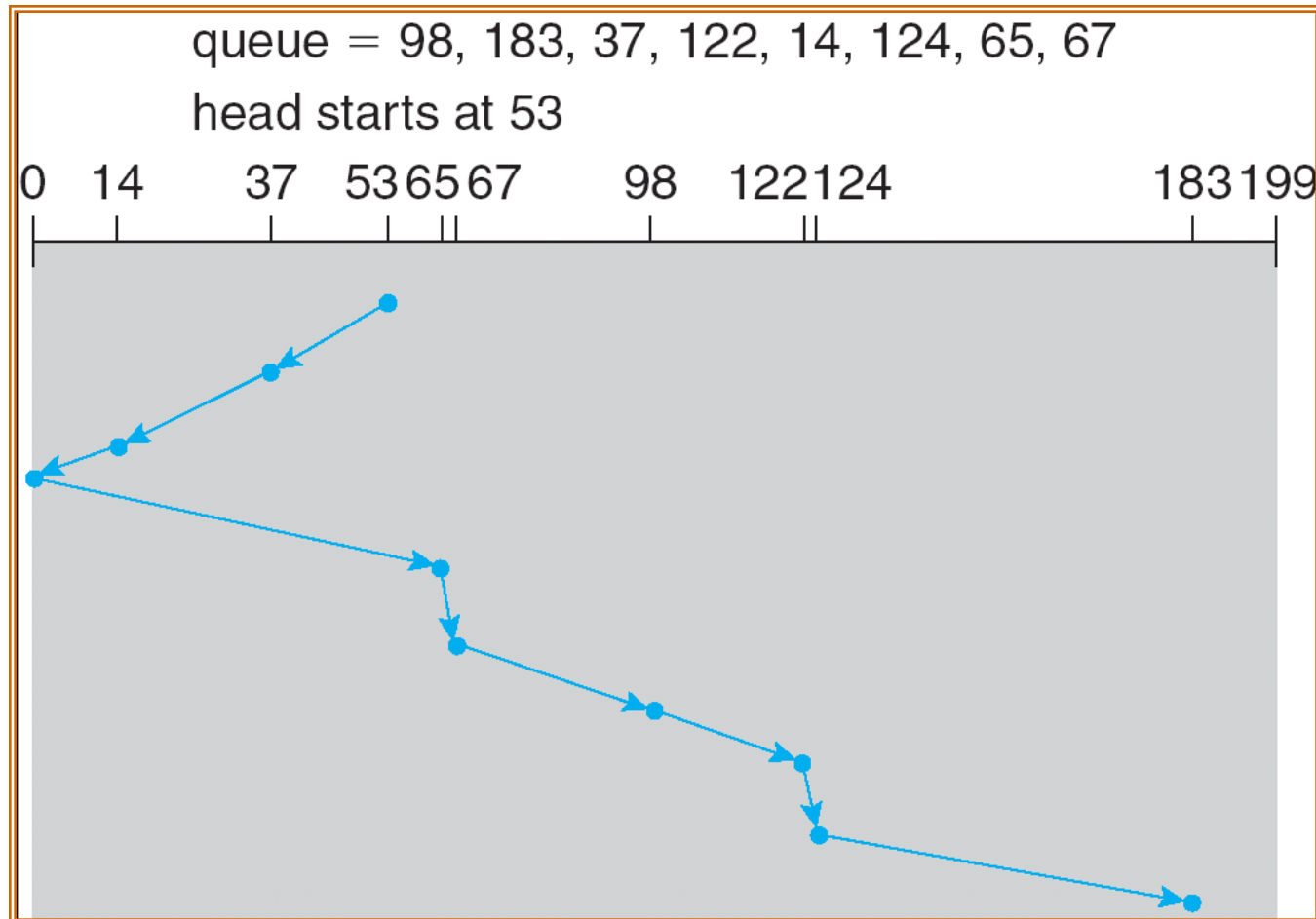
$$= 236$$

SCAN

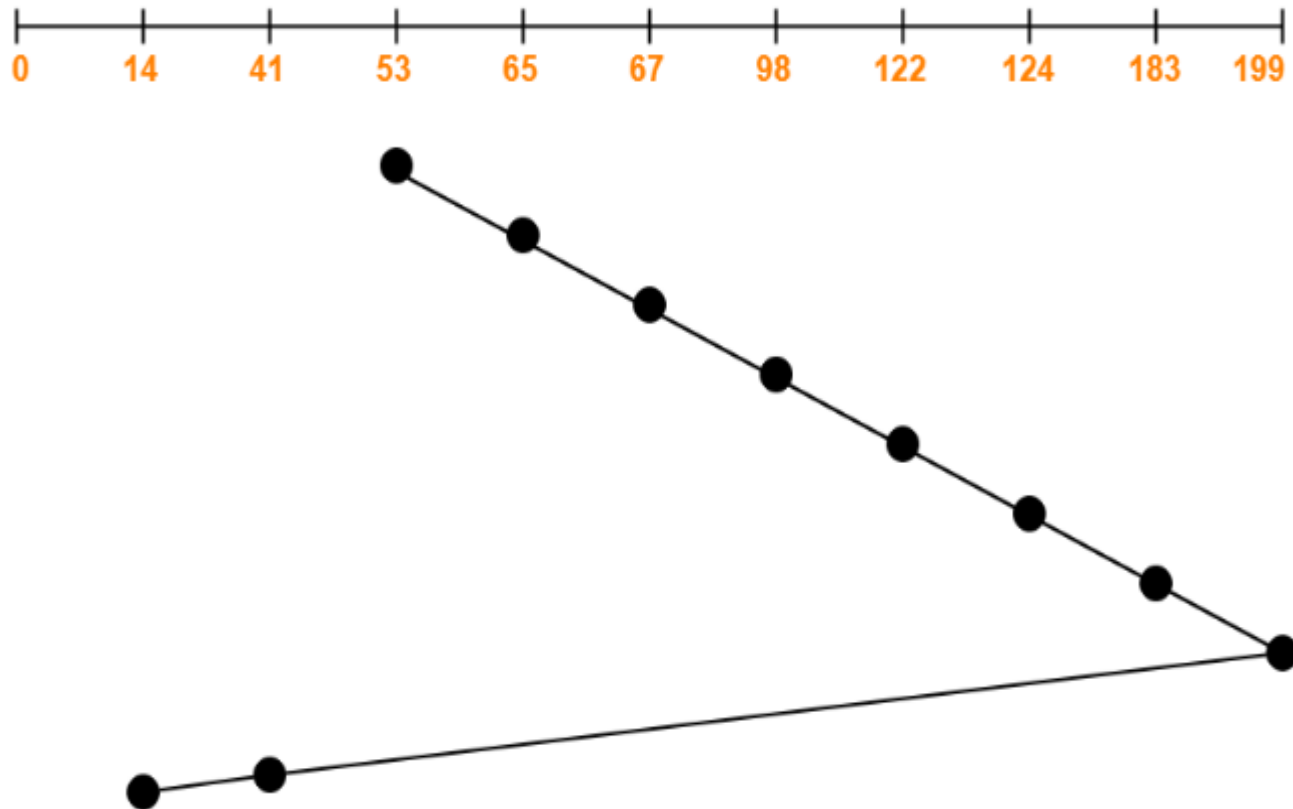
- As the name suggests, this algorithm scans all the cylinders of the disk back and forth.
- Head starts from one end of the disk and move towards the other end servicing all the requests in between.
- After reaching the other end, head reverses its direction and move towards the starting end servicing all the requests in between.
- The same process repeats.

SCAN / Elevator Disk Scheduling Algorithm

The disk arm starts at one end of the disk, and moves toward the other end, servicing requests until it gets to the other end of the disk, where the head movement is reversed and servicing continues.



Consider a disk queue with requests for I/O to blocks on cylinders 98, 183, 41, 122, 14, 124, 65, 67. The **SCAN** scheduling algorithm is used. The head is initially at cylinder number 53 moving towards larger cylinder numbers on its servicing pass. The cylinders are numbered from 0 to 199. The total head movement (in number of cylinders) incurred while servicing these requests is-----



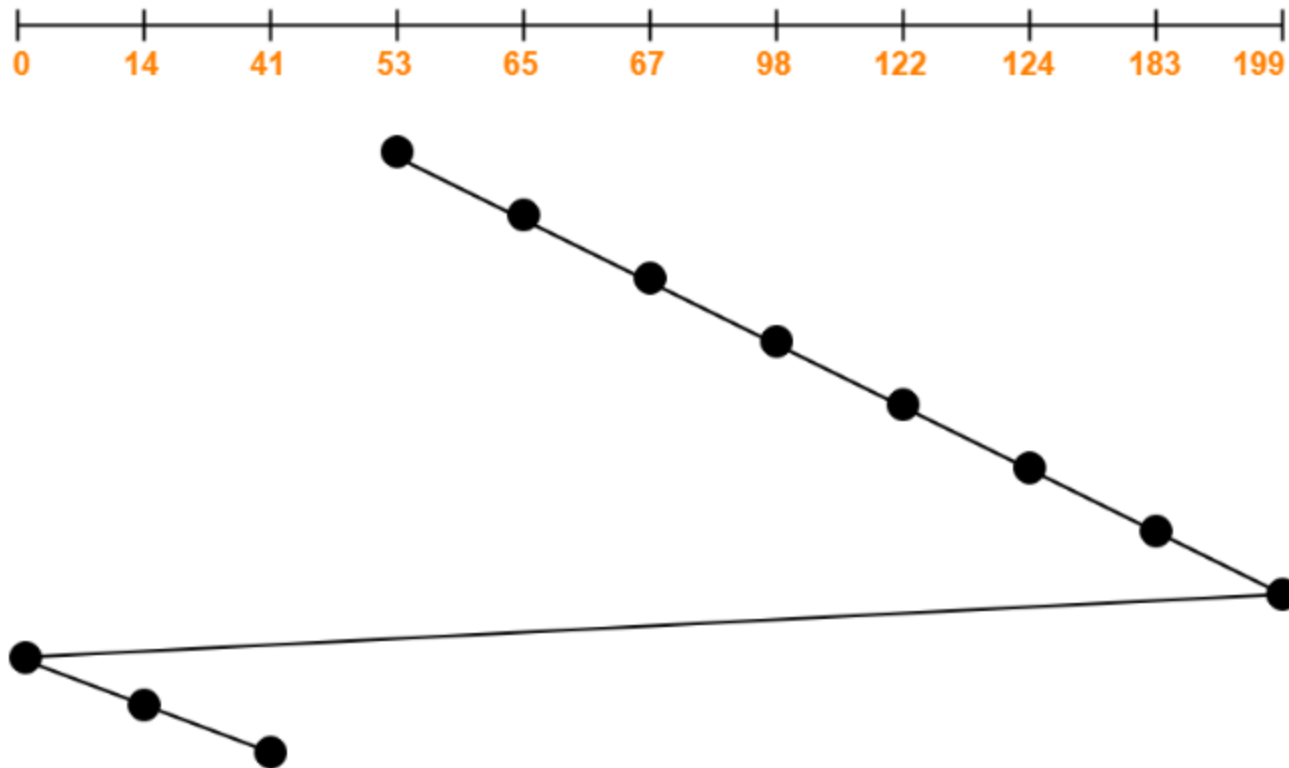
Total head movements incurred while servicing these requests

$$\begin{aligned} &= (65 - 53) + (67 - 65) + (98 - 67) + (122 - 98) + (124 - 122) + (183 - 124) + (199 - 183) + (199 - 41) + (41 - 14) \\ &= 12 + 2 + 31 + 24 + 2 + 59 + 16 + 158 + 27 \\ &= 331 \end{aligned}$$

C-SCAN

- Circular-SCAN Algorithm is an improved version of the SCAN Algorithm.
- Head starts from one end of the disk and move towards the other end servicing all the requests in between.
- After reaching the other end, head reverses its direction.
- It then returns to the starting end without servicing any request in between.
- The same process repeats.

Consider a disk queue with requests for I/O to blocks on cylinders 98, 183, 41, 122, 14, 124, 65, 67. The **C-SCAN** scheduling algorithm is used. The head is initially at cylinder number 53 moving towards larger cylinder numbers on its servicing pass. The cylinders are numbered from 0 to 199. The total head movement (in number of cylinders) incurred while servicing these requests is-----

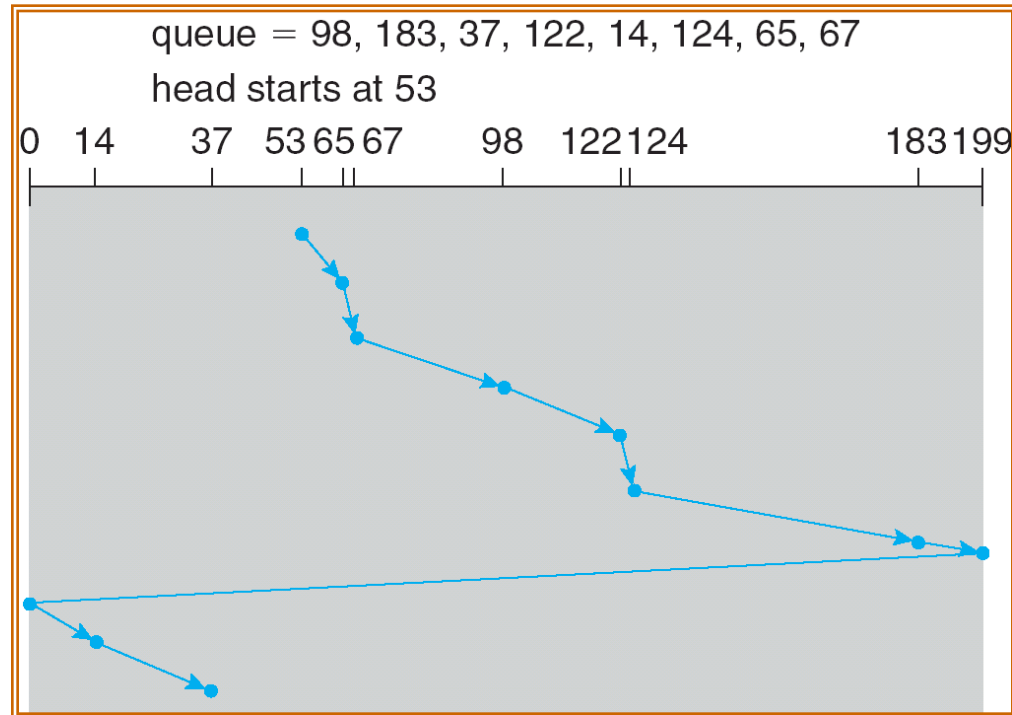


Total head movements incurred while servicing these requests

$$\begin{aligned}
 &= (65 - 53) + (67 - 65) + (98 - 67) + (122 - 98) + (124 - 122) + (183 - 124) + (199 - 183) + (14 - 0) + (41 - 14) \\
 &= 12 + 2 + 31 + 24 + 2 + 59 + 16 + 14 + 27 \\
 &= 187
 \end{aligned}$$

C-SCAN

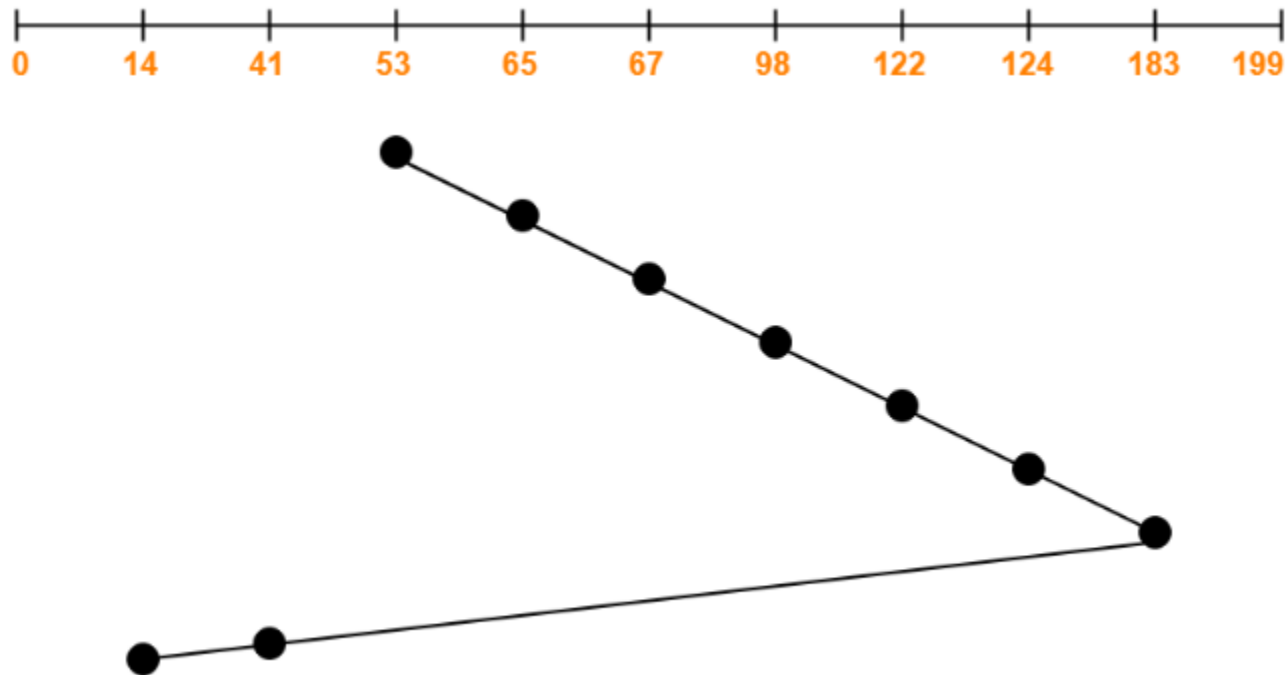
- Provides a more uniform wait time than SCAN.
- The head moves from one end of the disk to the other, servicing requests as it goes. When it reaches the other end, however, it immediately returns to the beginning of the disk, without servicing any requests on the return trip.
- Treats the cylinders as a circular list that wraps around from the last cylinder to the first one.



LOOK Disk Scheduling

- LOOK Algorithm is an improved version of the SCAN Algorithm.
- Head starts from the first request at one end of the disk and moves towards the last request at the other end servicing all the requests in between.
- After reaching the last request at the other end, head reverses its direction.
- It then returns to the first request at the starting end servicing all the requests in between.
- The same process repeats.

Consider a disk queue with requests for I/O to blocks on cylinders 98, 183, 41, 122, 14, 124, 65, 67. The LOOK scheduling algorithm is used. The head is initially at cylinder number 53 moving towards larger cylinder numbers on its servicing pass. The cylinders are numbered from 0 to 199. The total head movement (in number of cylinders) incurred while servicing these requests is-----

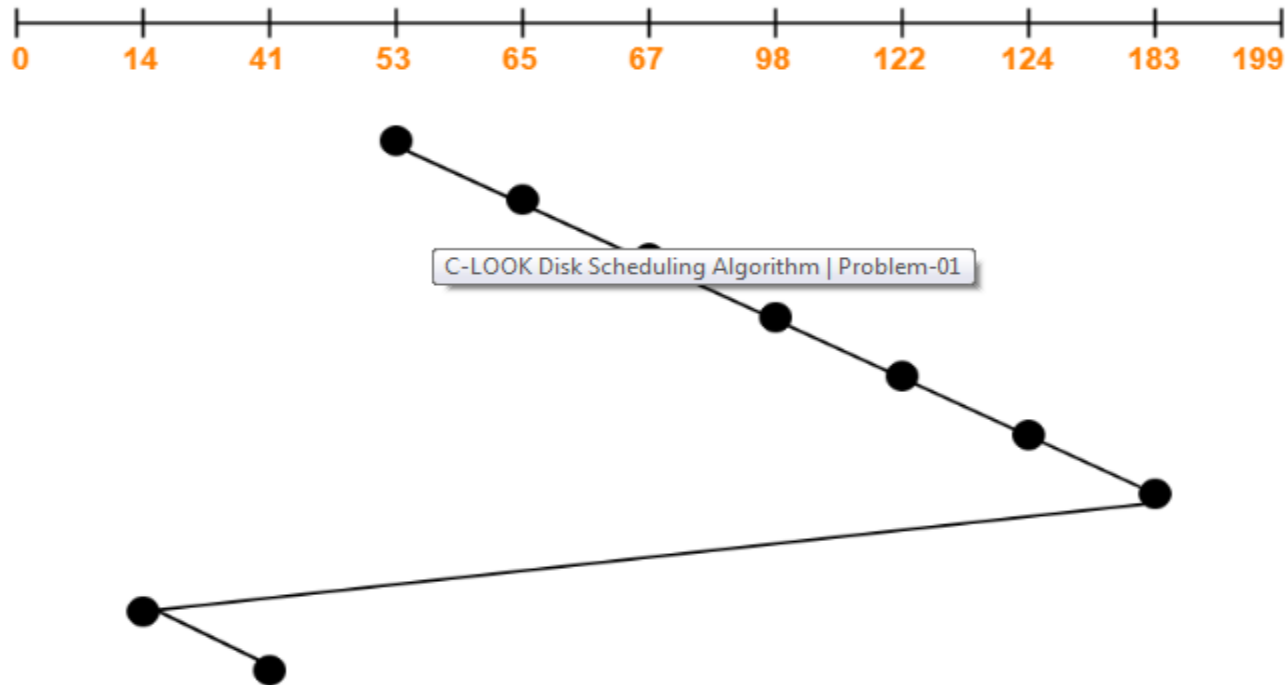


Total head movements incurred while servicing these requests
 $= (65 - 53) + (67 - 65) + (98 - 67) + (122 - 98) + (124 - 122) + (183 - 124) + (183 - 41) + (41 - 14) = 299$

C-LOOK

- Circular-LOOK Algorithm is an improved version of the LOOK Algorithm.
- Head starts from the first request at one end of the disk and moves towards the last request at the other end servicing all the requests in between.
- After reaching the last request at the other end, head reverses its direction.
- It then returns to the first request at the starting end without servicing any request in between.
- The same process repeats.

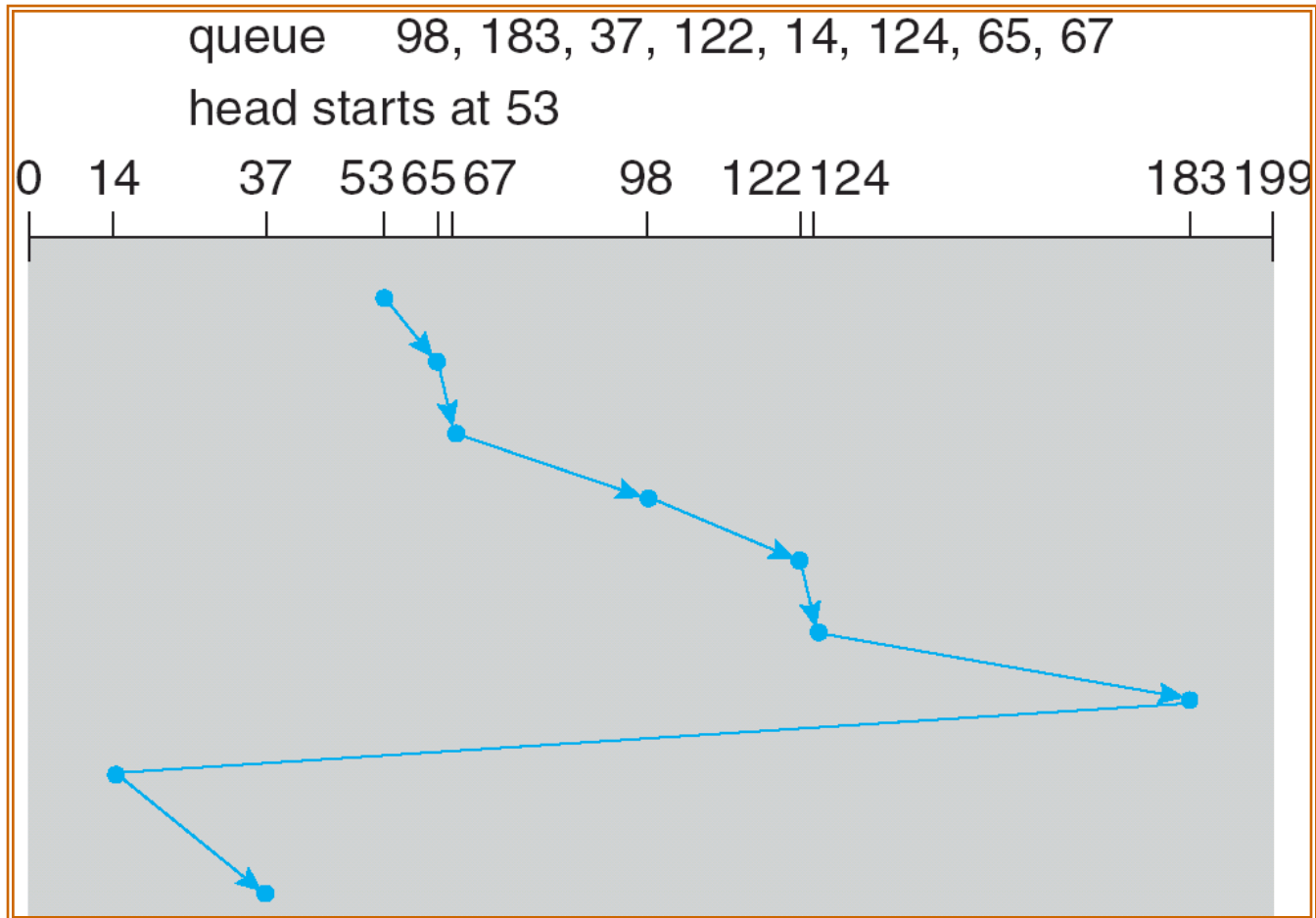
C-LOOK



$$\begin{aligned} & (65 - 53) + (67 - 65) + (98 - 67) + (122 - 98) + (124 - 122) + (183 - 124) + \\ & (183 - 14) + (41 - 14) \\ & = 12 + 2 + 31 + 24 + 2 + 59 + 169 + 27 \\ & = 326 \end{aligned}$$

C-LOOK

- Version of C-SCAN
- Arm only goes as far as the last request in each direction, then reverses direction immediately, without first going all the way to the end of the disk.



Selecting a Disk-Scheduling Algorithm

- **SSTF is common and has a natural appeal**
- **SCAN and C-SCAN perform better for systems that place a heavy load on the disk.**
- **Performance depends on the number and types of requests.**
- **Requests for disk service can be influenced by the file-allocation method.**
- **The disk-scheduling algorithm should be written as a separate module of the operating system, allowing it to be replaced with a different algorithm if necessary.**
- **Either SSTF or LOOK is a reasonable choice for the default algorithm.**



Disk Management

- Operating System is responsible for the disk management.

- Disk Formatting:

Two types..

- a) Physical formatting or low level formatting
- b) Logical formatting.

a) Physical formatting

- Disk must be formatted before storing a data
- Disk must be divided into sectors that the disk controller can read/write.
- Low level formatting fills the disk with a special data structure for each sector.
- Data structure consists of three fields: header, data area and trailer.
- Header and trailer contain information used by the disk controller.
- Sector no. and Error Correcting Codes(ECC) contained in the header and trailer.

Contd..

- For writing data to the sector-ECC is updated.
- For reading data from the sector- ECC is recalculated.
- If there is mismatch in stored and calculated value then data area is corrupted.
- Low level formatting is done at factory.

b) Logical Formatting:

- After disk is partitioned, logical formatting used.
- OS stores the initial file system data structures onto the disk.

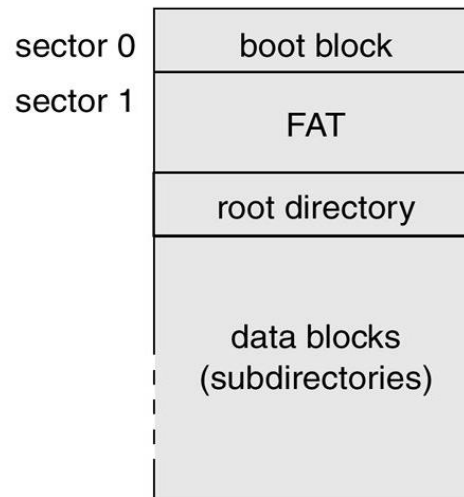
Boot Block

- When a computer system is powered up or rebooted, a program in read only memory executes.
- Diagnostics check is done first.
- Stage 0 boot program is executed.
- Boot program reads the first sector from the boot device into main memory.
- Boot sector is the first sector of the boot device and contains stage-1 boot program.
- May be boot sector will not contain a boot program
- PC booting from hard disk, the boot sector also contains a partition table.

contd..

- The code in the boot ROM instructs the disk controller to read the boot blocks into the memory and then starts executing that code.
- Full boot strap program is more sophisticated than the bootstrap loader in the boot ROM.

MS-DOS Disk Layout



Bad Blocks

- Data that resides on the bad blocks usually are lost. The controller maintains a list of bad blocks on the disk. The list is initialized during the low level format at the factory, and is updated over the life of the disk.
- Low level formatting also sets aside spare sectors not visible to the OS. The controller can be told to replace each bad sector logically with one of the spare sectors. This is known as **sector sparing** or **forwarding**.

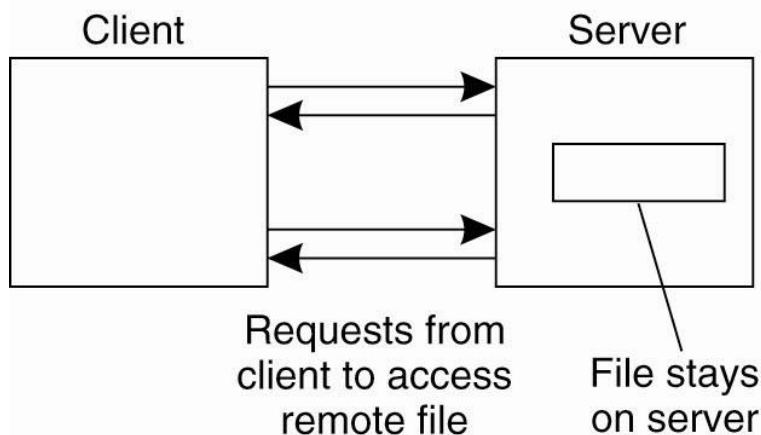
Distributed / Network File System (DFS) / (NFS)

- **Any File System that allows Access to Files from Multiple Hosts Sharing via a Computer Network.**
 - **Makes it Possible for Multiple Users on Multiple Machines to Share Files and Storage Resources.**
- **Client Nodes do not have Direct Access to the underlying Block Storage but interact over the Network using a Protocol.**
 - **Makes it possible to Restrict Access to the File System Depending on Access lists or Capabilities on both the Servers and the Clients, Depending on How the Protocol is Designed.**
- **Include facilities for Transparent replication and Fault tolerance.**
 - **When a limited no. of Nodes in a File System go offline, the System Continues to Work without Any Data loss.**

DFS / NFS

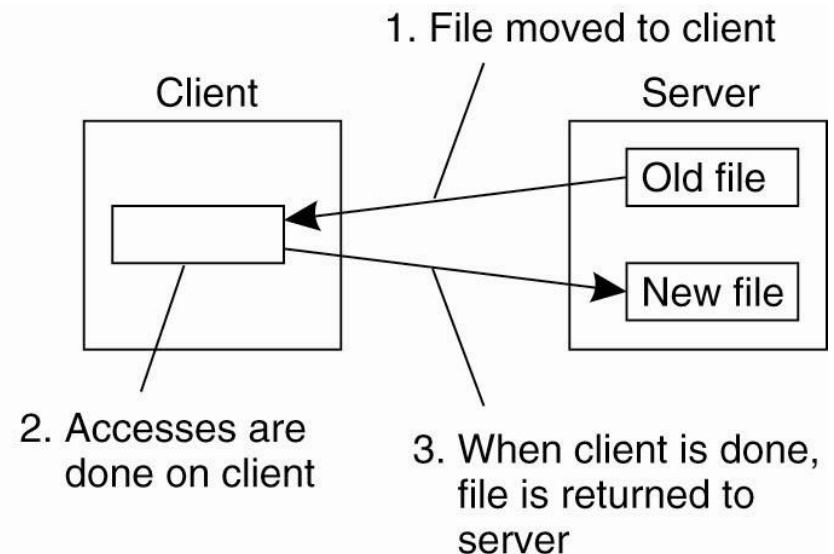
Requirements

- Heterogeneity
- Replication
- Transparency
- Consistency
- Concurrency
- Fault tolerance
- Security
- Efficiency



(a)

(a) The remote access model



(b)

(b) The upload / download model

Exam Questions



1. Describe the various *attributes of files*.
2. Write short note on *file sharing*.
3. Explain *Directory Structure*.
4. List the *file allocation methods*.
5. Explain the various *file allocation methods*, their *advantages and disadvantages*.
6. Discuss various *Disk Scheduling algorithms*.
7. Explain briefly about *disk scheduling algorithms* and its *types*.